

EE272 Homework 4

CNN HLS Accelerator Part 1

Report

Student Name 1: Berk Gokmen
Student Name 2: Edison Altamirano

Please answer the questions below in the space provided and include the code/reports indicated in your submission.

4 Catapult Tutorial

1. Please provide a screenshot of the read from empty channel error. (2 points)

```
[eraltam@rice-01 lab1]$ make tb1
/cad/mentor/2022.2/Mgc_home/bin/g++ -o /home/users/eraltam/272-new/hls/dnn-accelerator-hls-unoptimized/cxx/lab1/tb tb.cpp test_chan_assert.cpp -I/cad/mentor/2022.2/Mgc_home/shared/include -g /home/users/eraltam/272-new/hls/dnn-accelerator-hls-unoptimized/cxx/lab1/tb
Assert in file /cad/mentor/2022.2/Mgc_home/shared/include/ac_channel.h:229 Read from empty channel
tb: /cad/mentor/2022.2/Mgc_home/shared/include/ac_channel.h:175: static void ac_channel<T>::ac_assert(bool, const char*, int, const ac_channel_exception::code&) [with T = ac_int<4, false>]: Assertion `0' failed.
make: *** [tb1] Aborted (core dumped)
```

2. Please provide a screenshot showing the location of the bad channel read in gdb. (2 points)

```
(gdb) run
Starting program: /home/users/eraltam/272-new/hls/cxx/lab1/tb
Assert in file /cad/mentor/2022.2/Mgc_home/shared/include/ac_channel.h:229 Read from empty channel
tb: /cad/mentor/2022.2/Mgc_home/shared/include/ac_channel.h:175: static void ac_channel<T>::ac_assert(bool, const char*, int, const ac_channel_exception::code&) [with T = ac_int<4, false>]: Assertion `0' failed.

Program received signal SIGABRT, Aborted.
0x00007ffff7223387 in raise () from /lib64/libc.so.6
(gdb) up
#1 0x00007ffff7224a78 in abort () from /lib64/libc.so.6
(gdb) up
#2 0x00007ffff721c1a6 in __assert_fail_base () from /lib64/libc.so.6
(gdb) up
#3 0x00007ffff721c252 in __assert_fail () from /lib64/libc.so.6
(gdb) up
#4 0x000000000401b1f in ac_channel<ac_int<4, false> >::ac_assert (condition=false,
file=0x4039b8 "/cad/mentor/2022.2/Mgc_home/shared/include/ac_channel.h", line=229,
code=@0x7ffffffc8fc: ac_channel_exception::read_from_empty_channel)
at /cad/mentor/2022.2/Mgc_home/shared/include/ac_channel.h:175
175         assert(0);
(gdb) up
#5 0x0000000004026c2 in ac_channel<ac_int<4, false> >::fifo::fifo_ac_channel::read (this=0x608010)
at /cad/mentor/2022.2/Mgc_home/shared/include/ac_channel.h:229
229         AC_CHANNEL_ASSERT(!empty(), ac_channel_exception::read_from_empty_channel);
(gdb) up
#6 0x000000000401755 in ac_channel<ac_int<4, false> >::fifo::read (this=0x7ffffffca50)
at /cad/mentor/2022.2/Mgc_home/shared/include/ac_channel.h:402
402         inline T read() { return f->read(); }
(gdb) up
#7 0x00000000040152e in ac_channel<ac_int<4, false> >::read (this=0x7ffffffca50)
at /cad/mentor/2022.2/Mgc_home/shared/include/ac_channel.h:118
118         T read() { return chan.read(); }
(gdb)
```

3. Please provide a screenshot of the output with shift loss. (2 points)

```
[eraltam@rice-01 lab1]$ make tb2
/cad/mentor/2022.2/Mgc_home/bin/g++ -o /home/users/eraltam/272-new/hls/dnn-accelerator-hls-unoptimized/cxx
/lab1/tb tb.cpp test_shift_loss.cpp -I/cad/mentor/2022.2/Mgc_home/shared/include -g
/home/users/eraltam/272-new/hls/dnn-accelerator-hls-unoptimized/cxx/lab1/tb
Input = 0 wrap_behavior = 0, sat_behavior = 0, shift_behavior = 0
Input = 1 wrap_behavior = 1, sat_behavior = 1, shift_behavior = 2
Input = 2 wrap_behavior = 2, sat_behavior = 2, shift_behavior = 4
Input = 3 wrap_behavior = 3, sat_behavior = 3, shift_behavior = 6
Input = 4 wrap_behavior = 4, sat_behavior = 4, shift_behavior = 8
Input = 5 wrap_behavior = 5, sat_behavior = 5, shift_behavior = 10
Input = 6 wrap_behavior = 6, sat_behavior = 6, shift_behavior = 12
Input = 7 wrap_behavior = 7, sat_behavior = 7, shift_behavior = 14
Input = 8 wrap_behavior = 0, sat_behavior = 7, shift_behavior = 0
Input = 9 wrap_behavior = 1, sat_behavior = 7, shift_behavior = 2
Input = 10 wrap_behavior = 2, sat_behavior = 7, shift_behavior = 4
Input = 11 wrap_behavior = 3, sat_behavior = 7, shift_behavior = 6
Input = 12 wrap_behavior = 4, sat_behavior = 7, shift_behavior = 8
Input = 13 wrap_behavior = 5, sat_behavior = 7, shift_behavior = 10
Input = 14 wrap_behavior = 6, sat_behavior = 7, shift_behavior = 12
Input = 15 wrap_behavior = 7, sat_behavior = 7, shift_behavior = 14

Channel data = 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```

4. Please provide a screenshot of the output without shift loss. (2 points)

```
[eraltam@rice-01 lab1]$ make tb3
/cad/mentor/2022.2/Mgc_home/bin/g++ -o /home/users/eraltam/272-new/hls/dnn-accelerator-hls-unoptimized/cxx
/lab1/tb tb.cpp test.cpp -I/cad/mentor/2022.2/Mgc_home/shared/include -g
/home/users/eraltam/272-new/hls/dnn-accelerator-hls-unoptimized/cxx/lab1/tb
Input = 0 wrap_behavior = 0, sat_behavior = 0, shift_behavior = 0
Input = 1 wrap_behavior = 1, sat_behavior = 1, shift_behavior = 2
Input = 2 wrap_behavior = 2, sat_behavior = 2, shift_behavior = 4
Input = 3 wrap_behavior = 3, sat_behavior = 3, shift_behavior = 6
Input = 4 wrap_behavior = 4, sat_behavior = 4, shift_behavior = 8
Input = 5 wrap_behavior = 5, sat_behavior = 5, shift_behavior = 10
Input = 6 wrap_behavior = 6, sat_behavior = 6, shift_behavior = 12
Input = 7 wrap_behavior = 7, sat_behavior = 7, shift_behavior = 14
Input = 8 wrap_behavior = 0, sat_behavior = 7, shift_behavior = 16
Input = 9 wrap_behavior = 1, sat_behavior = 7, shift_behavior = 18
Input = 10 wrap_behavior = 2, sat_behavior = 7, shift_behavior = 20
Input = 11 wrap_behavior = 3, sat_behavior = 7, shift_behavior = 22
Input = 12 wrap_behavior = 4, sat_behavior = 7, shift_behavior = 24
Input = 13 wrap_behavior = 5, sat_behavior = 7, shift_behavior = 26
Input = 14 wrap_behavior = 6, sat_behavior = 7, shift_behavior = 28
Input = 15 wrap_behavior = 7, sat_behavior = 7, shift_behavior = 30

Channel data = 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```

5. Please explain how the shift loss was fixed. (2 points)

The shift loss occurred because `data0` was defined as a 4-bit unsigned type (`ac_int<4,false>`), so left shifting by one bit caused the most significant bit to overflow and be discarded when the result exceeded the available bit width. This was fixed by casting `data0` to a wider type (`ac_int<5,false>`) before performing the shift, ensuring that the shifted value fits within the variable and preventing loss of the MSB.

In `tb2`, the output shows a wrap-around behavior (for example, 14 $\rightarrow$ 0) because the left shift is performed on a 4-bit variable, causing the most significant bit to be lost when the value exceeds the available bit width. In `tb3`, the output shows correct values (up to 30) because the data is cast to a wider type before the shift operation, allowing the result to fit without losing any bits.

7.2 Systolic Array

1. How many cycles does the systolic array run to produce a valid output tile? (2 points)

The total cycles are the input fill latency ($IC0-1$) plus the per-pixel streaming time (`num_pixels`) plus the output drain latency ($OC0-1$). Therefore, we have that $step_bound = num_pixels + (IC0-1) + (OC0-1)$ cycles. With $OX0*OY0=49$, $IC0=16$, $OC0=16$, this is $49 + 15 + 15 = 79$ cycles.

2. When will the first valid output pixel be output to the output skew registers? (2 points)

The first output pixel becomes valid after both pipelines have finished filling, so the first valid output appears at $valid_start = IC0 + OC0 - 2$ cycles. With $IC0=16$ and $OC0=16$, $valid_start = 30$.

3. How many weight lines ($OC0$ weights) will be loaded during the tile computation? (2 points)

Weights are loaded only during the ramp-up ($step < IC0$), so the number of weight lines loaded per run is $IC0 = 16$.

4. How many input pixel lines ($IC0$ pixels) will be loaded during the tile computation? (2 points)

Inputs are loaded for each output pixel ($step < num_pixels$), so the number of input pixel lines loaded per run is $num_pixels = OX0*OY0 = 49$.

5. How many partial sums ($IC0$ partial sums) will be loaded during the tile computation? (2 points)

Partial sums are loaded only when this is not the first accumulation pass ($first==0$). Then one psum line is loaded per pixel, so $num_pixels = 49$ (otherwise it would be 0 when $first==1$).

6. When should we use 0 instead of the output of the accumulation buffer for partial sums into the systolic array? (2 points)

We use 0 only on the first accumulation pass for a tile, when `first == 1` and `ic1 = 0`, and during the flush phase (`step >= num_pixels`). For all subsequent passes, the accumulation buffer output is used to continue accumulating partial sums.

8 Implementation Autograding

1. Please provide a screenshot of the autograder passing all ResNet-18 layers for `c_test`. We will also run this ourselves. (48 points)

```
teraltam@iron-01 dnn-accelerator-hls-unoptimized]$ python3 autograder.py c_test -l all
Tests being run: ['test_c_test']
Running c_test with layer params: ./layers/resnet_conv1_params.json
Test passed!
Running c_test with layer params: ./layers/resnet_conv2_x_params.json
Test passed!
Running c_test with layer params: ./layers/resnet_conv3_1_params.json
Test passed!
Running c_test with layer params: ./layers/resnet_conv3_x_params.json
Test passed!
Running c_test with layer params: ./layers/resnet_conv4_1_params.json
Test passed!
Running c_test with layer params: ./layers/resnet_conv4_x_params.json
Test passed!
Running c_test with layer params: ./layers/resnet_conv5_1_params.json
Test passed!
Running c_test with layer params: ./layers/resnet_conv5_x_params.json
Test passed!
Tests passed: 8
Tests run: 8
elapsed time: 802.5488083362579
```

2. Please provide a screenshot of the autograder passing all small layers for the rtl_test. We will also run this ourselves. (30 points)

```
python3 rtl_test
[bgokmen@iron-06 dnn-accelerator-hls-unoptimized]$ python3 autograder.py rtl_test -l small
Tests being run: ['test_rtl_test']
Running rtl_test with layer params: ./layers/small_layer1.json
Test passed!

Running rtl_test with layer params: ./layers/small_layer2.json
Test passed!

Running rtl_test with layer params: ./layers/small_layer3.json
Test passed!

Tests passed: 3
Tests run: 3
elapsed time: 4691.045314073563
```